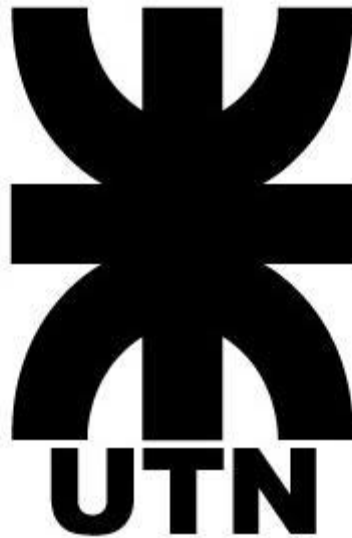


Universidad Tecnológica Nacional



Cátedra: Algoritmos Genéticos

4º Año Ingeniería en Sistemas de Información

Trabajo Práctico N°2

Comisión: 303

Grupo : 14

Integrantes:

- RIBOTTA, Tomás	52309	tomasribotta20@gmail.com
- MAZALAN, Ariel	52867	arielmazalan11@gmail.com
- GIACONE, Alessandro	52664	alef.giacone@gmail.com
- BUSNADIEGO, Martin	53020	martinbusnaisi@gmail.com

1. Introducción Teórica	3
1.1. Optimización Combinatoria	3
1.2. El Problema de la Mochila	3
1.3. Formalización Matemática	3
1.4. Complejidad del Problema	4
2. Métodos de Resolución	4
2.1. Búsqueda Exhaustiva	4
2.2. Algoritmo Heurístico Greedy (Goloso)	5
2.3. Comparación entre Enfoques	5
3. Descripción del Problema	6
3.1. Datos: Ejercicios 1 y 2	6
3.2. Datos: Ejercicio 3	7
4. Ejercicio 1: Búsqueda Exhaustiva (10 objetos)	7
4.1. Representación del espacio de soluciones	7
4.2. Codificación en Python	7
4.3. Resultados	8
5. Ejercicio 2: Algoritmo Greedy (10 objetos)	8
5.1. Cálculo del ratio valor/volumen	8
5.2. Codificación en Python	9
5.3. Resultados	10
5.4. Comparación entre greedy y la Búsqueda Exhaustiva	10
6. Ejercicio 3: Problema de la Mochila con Restricción de Peso	11
6.1. Resolución por Algoritmo Goloso	11
6.2. Resolución Exhaustiva	11
6.3. Análisis Comparativo y Conclusiones del Ejercicio	12
7. Conclusiones Generales	13
8. Anexos: Código Fuente en Python	14

1. Introducción Teórica

1.1. Optimización Combinatoria

A pesar del desarrollo científico de los últimos años, aún quedan muchos problemas reales de optimización con una gran importancia práctica que no han sido resueltos. Dentro de este grupo están los que se caracterizan porque sus espacios solución están formados por subconjuntos de números naturales y, por dicha razón, se los llama problemas de optimización combinatoria.

La optimización combinatoria es una rama de la optimización en matemáticas aplicadas y en ciencias de la computación, relacionada con la investigación de operaciones, la teoría de algoritmos y la teoría de la complejidad computacional. También está relacionada con otros campos como la inteligencia artificial e ingeniería de software.

Los algoritmos de optimización combinatoria resuelven instancias de problemas que se creen difíciles en general, explorando el espacio de soluciones (usualmente grande) para estas instancias. Lo logran reduciendo el tamaño efectivo del espacio y explorando el espacio de búsqueda eficientemente.

1.2. El Problema de la Mochila

El problema de la mochila consiste en elegir, de entre un conjunto de **N** elementos (cada uno con un valor $\$i$ y un volumen V_i , aquellos que puedan ser cargados en una mochila de volumen **V**, de manera que el valor total obtenido sea máximo.

El conjunto solución debe respetar la restricción de capacidad:

$$\sum_{i \in S} V_i \leq V$$

y maximizar:

$$\sum_{i \in S} \$i$$

1.3. Formalización Matemática

Dado un conjunto de **N** elementos, cada uno caracterizado por:

- **V_i** : volumen (o peso) del elemento i
- **$\$i$** : valor del elemento i

Y una mochila de capacidad máxima **V**, se busca el subconjunto **S** $\subseteq \{1, 2, \dots, N\}$ tal que:

$$\text{Maximizar } f(S) = \sum_{i \in S} \$i$$

$$\text{sujeto a } \sum_{i \in S} V_i \leq V$$

1.4. Complejidad del Problema

Planteando el problema de la mochila con un procedimiento exhaustivo que evalúa todos los subconjuntos posibles, la solución encontrada tendría la importancia de ser única (óptima global). Sin embargo, el número de subconjuntos a evaluar crece exponencialmente con N.

Si el conjunto es de **N** elementos, el espacio solución tendrá **2^N** posibles subconjuntos. Por ejemplo:

N (elementos)	2 ^N (subconjuntos)	Tiempo estimado (1M eval/seg)
10	1.024	Milisegundos
20	1.048.576	~1 segundo
40	~1,1 × 10 ¹²	~2 semanas
60	~1,15 × 10 ¹⁸	~365 siglos

Esto demuestra que el problema es manejable cuando N es reducido, pero a medida que N crece el tiempo aumenta rápidamente, y para valores grandes de N no hay forma de resolverlo exhaustivamente en tiempo razonable.

2. Métodos de Resolución

2.1. Búsqueda Exhaustiva

La búsqueda exhaustiva es un método exacto que garantiza encontrar la solución óptima del problema. Para implementarlo, no se descartan opciones sobre la marcha (lo cual lo acercaría a un método de tipo burbuja), sino que se generan y almacenan absolutamente todos los subconjuntos posibles. Dado que cada objeto puede estar o no en la mochila, se utiliza una representación binaria donde 1 bit representa un objeto. Con N objetos, se arman los 2^N subconjuntos posibles iterando sobre los números binarios. Una vez que toda la estructura de subconjuntos está generada en memoria, se procede a evaluar cada uno, calculando su

volumen y valor total, para finalmente trasladar a otra estructura únicamente a aquel que maximice el valor sin exceder la capacidad máxima.

2.2. Algoritmo Heurístico Greedy (Goloso)

Cuando el problema es necesario resolverlo inmediatamente o cuando N es grande, se recurre a métodos heurísticos: algoritmos que no garantizan la solución óptima, pero que encuentran soluciones factibles (que respetan las restricciones) de buena calidad en un tiempo de cálculo razonable.

Una clase particular de heurísticas son los métodos constructivos, que agregan componentes individuales a la solución hasta que se obtiene una solución factible. El algoritmo greedy (goloso o devorador) es un ejemplo de estos métodos. Construye la solución paso a paso eligiendo, en cada paso, la opción que ofrece el máximo beneficio inmediato.

Para el problema de la mochila, el criterio greedy consiste en:

1. Calcular el ratio $\$i / Vi$ (valor por unidad de volumen) para cada elemento.
2. Ordenar los elementos de mayor a menor ratio.
3. Cargar la mochila en ese orden hasta que no entre ningún elemento más.
4. Si al detenerse, algún elemento posterior (de menor ratio) cabe en el espacio restante, incluirlo.

Importante: el algoritmo greedy no garantiza la solución óptima global. En algunos casos puede diferir del resultado exhaustivo, como se analizará luego.

2.3. Comparación entre Enfoques

Característica	Búsqueda Exhaustiva	Algoritmo Greedy
Garantía de optimalidad	Sí	No
Complejidad temporal	$O(2^N)$	$O(N \log N)$
Aplicable para N grande	No	Sí
Facilidad de implementación	Media	Alta

La evaluación de ambos métodos se centra en tres ejes:

- **Optimalidad:** El método exhaustivo siempre encuentra la solución óptima; el algoritmo Greedy puede encontrarla o quedar atrapado en un óptimo local.

- **Tiempo de ejecución:** El algoritmo Greedy resuelve el problema en una complejidad temporal $O(N \log N)$ (dominada por el ordenamiento inicial), haciéndolo casi instantáneo. El método exhaustivo posee una complejidad exponencial $O(2^N)$, lo que incrementa masivamente el tiempo de ejecución a medida que crece el tamaño del problema.
- **Uso de memoria:** El enfoque exhaustivo requiere almacenar previamente todas las combinaciones posibles, lo que consume una cantidad significativa de recursos para un N grande, mientras que el heurístico requiere un uso de memoria mínimo.

3. Descripción del Problema

3.1. Datos: Ejercicios 1 y 2

Se dispone de 10 objetos con sus respectivos volúmenes y valores, y una mochila de capacidad máxima **4200 cm³**:

Objeto	Volumen (cm ³)	Valor (\$)
1	150	20
2	325	40
3	600	50
4	805	36
5	430	25
6	1200	64
7	770	54
8	60	18
9	930	46
10	353	28

Volumen máximo soportado por la mochila: 4200 cm³

3.2. Datos: Ejercicio 3

Se trabaja con 3 elementos con restricción de **peso** (en lugar de volumen) y una mochila de capacidad **3000 grs**:

Elemento	Peso (grs)	Valor (\$)
1	1800	72
2	600	36
3	1200	60

Capacidad máxima de la mochila: 3000 grs

4. Ejercicio 1: Búsqueda Exhaustiva (10 objetos)

4.1. Representación del espacio de soluciones

Con $N = 10$ objetos, el espacio de soluciones está compuesto por $2^{10} = 1024$ subconjuntos posibles. Cada subconjunto se representa como un número binario de 10 bits: si el bit i es 1, el objeto i se incluye en la mochila; si es 0, no se incluye.

Para cada subconjunto se evalúa:

- **Volumen total**: suma de los volúmenes de los objetos incluidos.
- **Valor total**: suma de los valores de los objetos incluidos.

Un subconjunto es válido si su volumen total no supera los 4200 cm³. Entre todos los válidos, se selecciona el de mayor valor.

4.2. Codificación en Python

Para implementar la búsqueda exhaustiva, la función `busqueda_exhaustiva` se estructuró en dos grandes bloques secuenciales:

1. **Generación y Almacenamiento de Subconjuntos**: Primero se calcula la cantidad total de subconjuntos posibles (2^N). Luego, mediante un bucle iterativo, se utiliza la representación binaria de cada número (de 0 a 2^N-1) y operaciones de desplazamiento de bits (`((numero >> i) & 1)` para determinar si un objeto pertenece o no al subconjunto

actual (1 bit por objeto). Durante esta fase no se descarta ninguna opción ni se evalúa el volumen máximo; simplemente se calcula el volumen y valor de cada combinación y se guardan absolutamente todas en una única estructura de datos (todos_los_subconjuntos).

2. **Evaluación y Selección del Óptimo:** Una vez que el espacio de soluciones está completamente generado y almacenado en memoria, se procede a recorrer dicha estructura. Para cada subconjunto, se verifica si es una solución factible (es decir, si su volumen no excede la capacidad máxima de la mochila). Entre todos los subconjuntos válidos, se compara el valor obtenido y se almacena en una variable (mejor_subconjunto) aquella combinación que maximice la ganancia. Finalmente, la función retorna la solución óptima.

4.3. Resultados

Tras ejecutar el algoritmo de búsqueda exhaustiva, el programa generó en memoria y evaluó la totalidad del espacio de soluciones, compuesto por $2^{10} = 1024$ subconjuntos posibles (cuyos detalles se encuentran exportados en el archivo ejercicio1_subconjuntos.csv).

De todos los subconjuntos generados, la **solución óptima global** encontrada es la siguiente:

- **Objetos seleccionados:** {1, 2, 3, 5, 6, 7, 8, 10}
- **Volumen total:** 3888 cm³
- **Valor total:** \$299

El resultado obtenido nos da la certeza matemática de ser el óptimo global del problema para este conjunto de datos. Al no descartar ninguna combinación, el algoritmo exploró todas las alternativas posibles. Pudo determinar que, dejando fuera únicamente a los objetos 4 y 9 (que ocupan mucho volumen en relación a su valor), se logra empaquetar el resto de los elementos maximizando la ganancia en \$299 y respetando el límite físico, dejando un espacio sobrante de solo 312 cm³ en la mochila.

Dado que el tamaño del problema es relativamente pequeño ($N=10$), el tiempo de ejecución del algoritmo es del orden de los milisegundos (aprox. 0.001 a 0.003 segundos, dependiendo del procesador), lo que demuestra que para instancias reducidas, la búsqueda exhaustiva es un enfoque perfectamente viable.

5. Ejercicio 2: Algoritmo Greedy (10 objetos)

5.1. Cálculo del ratio valor/volumen

El primer paso del algoritmo greedy es calcular el ratio $\$i / Vi$ para cada objeto y ordenarlos de mayor a menor:

Objeto	Volumen (cm ³)	Valor (\$)	Ratio (\$/cm ³)
8	60	18	0.300
1	150	20	0.1333
2	325	40	0.1231
3	600	50	0.0833
10	353	28	0.0793
7	770	54	0.0701
5	430	25	0.0581
6	1200	64	0.0533
9	930	46	0.0395
4	805	36	0.0337

5.2. Codificación en Python

Para la implementación del algoritmo heurístico Greedy, la función `algoritmo_greedy` se estructuró siguiendo el método constructivo detallado en la introducción teórica. El proceso en el código consta de dos etapas principales:

1. **Cálculo de Proporciones y Ordenamiento:** En primera instancia, se recorre el conjunto de objetos para calcular el ratio o beneficio inmediato de cada uno, dividiendo su valor por su volumen (\$/cm³). Cada objeto se guarda en una lista junto con su índice original, volumen, valor y este nuevo ratio calculado. A continuación, se ordena esta lista en forma descendente (de mayor a menor) basándose estrictamente en el ratio obtenido.
2. **Construcción de la Solución:** Una vez ordenada la lista, se la recorre secuencialmente. Para cada objeto, se evalúa si la suma del volumen acumulado hasta el momento más el volumen del objeto actual es menor o igual a la capacidad máxima de la mochila (4200 cm³). Si el objeto entra, se incluye en la lista de elementos seleccionados y se actualizan los acumuladores de volumen y valor. Si el objeto supera la capacidad restante, simplemente se lo descarta y el ciclo continúa con el siguiente elemento de la lista para intentar aprovechar cualquier espacio ocioso que haya quedado.

5.3. Resultados

Al ejecutar el algoritmo Greedy con los 10 objetos, el programa primero estableció el orden de prioridad según el ratio de valor por unidad de volumen y luego procedió a llenar la mochila (el detalle de este ordenamiento se exportó en el archivo ejercicio2_orden_greedy.csv).

Los resultados arrojados por el algoritmo son:

- **Objetos seleccionados (en orden de inserción):** {8, 1, 2, 3, 10, 7, 5, 6}
- **Volumen total:** 3888 cm³
- **Valor total:** \$299

El algoritmo como ya lo mencionamos construyó la solución insertando los objetos de manera secuencial siguiendo estrictamente el orden de sus ratios. Logró incluir los primeros 8 elementos de la lista ordenada sin problemas. Al intentar ingresar el noveno elemento (el Objeto 9), el volumen acumulado pasaría a ser de 4818 cm³, superando el límite de 4200 cm³, por lo que fue descartado. Lo mismo ocurrió con el último elemento de la lista (el Objeto 4), cuyo volumen de 805 cm³ no cabía en el espacio restante de 312 cm³. Al detenerse la ejecución tras evaluar la lista completa, el algoritmo heurístico cerró su conjunto solución factible con un valor total de \$299.

5.4. Comparación entre greedy y la Búsqueda Exhaustiva

Al contrastar los resultados de ambos métodos para este conjunto de 10 objetos, se observa que el algoritmo heurístico Greedy logró alcanzar la solución óptima global. Ambos enfoques seleccionaron exactamente el mismo subconjunto de elementos, obteniendo un valor máximo de \$299 y ocupando 3888 cm³ de la mochila. En este escenario particular, el criterio de selección local (ratio valor/volumen) guió exitosamente al algoritmo hacia la mejor combinación posible, demostrando que las heurísticas pueden encontrar el óptimo en ciertas instancias del problema.

La diferencia fundamental entre ambas aproximaciones radica en los tiempos de ejecución, evidenciando las distintas naturalezas de sus métodos de resolución:

- **Tiempo Búsqueda Exhaustiva:** 0.000704 segundos.
- **Tiempo Algoritmo Greedy:** 0.000007 segundos.

El método heurístico resultó ser aproximadamente 100 veces más rápido. Esta brecha de rendimiento se explica por la complejidad computacional teórica de cada algoritmo. La búsqueda exhaustiva debió construir en memoria y evaluar 1024 combinaciones (2^{10}), operando bajo una complejidad temporal de $O(2^N)$. En cambio, el algoritmo Greedy logró su objetivo simplemente ordenando una lista de 10 elementos y recorriéndola de forma lineal, lo que representa una complejidad de $O(N \log N)$.

Si bien para este conjunto reducido de datos el tiempo del método exhaustivo sigue siendo del orden de los microsegundos, esta comparación empírica demuestra de forma práctica cómo la explosión combinatoria penaliza a los métodos exactos, haciendo indispensables a las heurísticas constructivas cuando el volumen de datos crece.

6. Ejercicio 3: Problema de la Mochila con Restricción de Peso

6.1. Resolución por Algoritmo Goloso

Para aplicar el método heurístico Greedy en este escenario con restricción de peso, el algoritmo procedió primero a calcular la proporción de valor por unidad de peso (\$/gr) para cada elemento, ordenándolos de forma decreciente de la siguiente manera:

1. **Elemento 2:** $\$36 / 600 \text{ grs} = 0.0600 \text{ \$/gr}$
2. **Elemento 3:** $\$60 / 1200 \text{ grs} = 0.0500 \text{ \$/gr}$
3. **Elemento 1:** $\$72 / 1800 \text{ grs} = 0.0400 \text{ \$/gr}$

Con la lista priorizada según el mejor beneficio inmediato, el proceso de selección paso a paso fue el siguiente:

- **Paso 1:** Se evalúa el **Elemento 2** (ratio máximo). Su peso es de 600 grs. Como es menor a la capacidad total de la mochila (3000 grs), se incluye.
 - *Espacio disponible restante:* 2400 grs.
- **Paso 2:** Se evalúa el **Elemento 3**. Su peso de 1200 grs cabe perfectamente en el espacio sobrante, por lo que también se incluye en la solución.
 - *Espacio disponible restante:* 1200 grs.
- **Paso 3:** Se intenta ingresar el **Elemento 1**. Sin embargo, su peso de 1800 grs excede el espacio disponible de 1200 grs, por lo que el algoritmo lo descarta.

Resultado final del Algoritmo Greedy:

- **Elementos seleccionados:** {2, 3}
- **Peso total:** 1800 grs
- **Valor total:** \$96
- **Tiempo de ejecución:** 0.00000271 segundos.

6.2. Resolución Exhaustiva

Con $N = 3$ elementos, el espacio de soluciones tiene $2^3 = 8$ **subconjuntos** posibles, perfectamente enumerables a mano:

Subconjunto	Elementos	Peso total (grs)	Valor total (\$)	¿Válido?
000	Ninguno	0	0	✓
001	{3}	1200	60	✓
010	{2}	600	36	✓
011	{2,3}	1800	96	✓
100	{1}	1800	72	✓
101	{1,3}	3000	132	✓
110	{1,2}	2400	108	✓
111	{1,2,3}	3600	168	×

Resultado final de la Búsqueda Exhaustiva:

Tras evaluar todas las combinaciones factibles, el algoritmo determinó que la solución óptima global corresponde al subconjunto {1, 3}. Esta combinación maximiza el beneficio obteniendo un valor total de \$132 y ocupando exactamente el 100% de la capacidad de la mochila (3000 grs). El tiempo de ejecución para evaluar todo el espacio de soluciones fue de 0.00000488 segundos.

6.3. Análisis Comparativo y Conclusiones del Ejercicio

Al comparar los resultados obtenidos por ambos algoritmos para este conjunto de 3 elementos, se evidencia claramente que el algoritmo Greedy NO logró encontrar la solución óptima global. Existe una diferencia sustancial de \$36 a favor del método exacto (\$132 de la Búsqueda Exhaustiva contra los \$96 del Greedy).

¿Por qué falló el método Greedy? El algoritmo goloso toma decisiones basándose exclusivamente en el beneficio local e inmediato, seleccionando paso a paso los elementos con mejor ratio valor/peso. La falla se produce por la siguiente secuencia:

1. El algoritmo incluyó primero el **Elemento 2** por tener el ratio más alto (0.0600 \$/gr). Sin embargo, al ser un objeto liviano (600 grs), consumió poco volumen pero forzó una decisión irreversible.
2. Luego incluyó el **Elemento 3** (1200 grs).
3. Al intentar ingresar el **Elemento 1** (el de mayor valor absoluto, pero con el peor ratio), se encontró con que el espacio restante (1200 grs) era insuficiente para albergar sus

1800 grs. El elemento fue descartado, dejando la mochila con un 40% de espacio totalmente desaprovechado (1200 grs ociosos).

En contraste, la Búsqueda Exhaustiva evaluó el panorama completo. Al no realizar descartes prematuros, el método exacto detectó que sacrificar el objeto con el mejor ratio individual (Elemento 2) era estratégicamente necesario para poder combinar los Elementos 1 y 3. Esta combinación ocupó el 100% de la capacidad máxima de la mochila (3000 grs) y entregó el máximo valor absoluto posible (\$132).

Este caso demuestra en la práctica que el criterio de selección local de una heurística constructiva no siempre conduce a la mejor combinación global. Al ser incapaz de retroceder, deshacer decisiones pasadas o evaluar el espacio ocioso que va dejando, el algoritmo Greedy puede quedar atrapado en un óptimo local. Por lo tanto, aunque los algoritmos heurísticos son invaluable por su velocidad computacional ante problemas de gran tamaño (N grande), se asume el riesgo de sacrificar la optimalidad de la solución.

7. Conclusiones Generales

A partir de la resolución analítica y práctica del Problema de la Mochila, se pueden establecer las siguientes conclusiones fundamentales sobre los métodos de optimización combinatoria:

El dilema entre optimalidad y complejidad computacional: Quedó evidenciado que la búsqueda exhaustiva es el único enfoque que garantiza matemáticamente la solución óptima global al no descartar ninguna combinación. No obstante, su complejidad exponencial de $O(2^N)$ la convierte en una alternativa completamente inviable cuando la cantidad de elementos (N) es grande, por esa misma razón dejaremos este tipo de solución para problemas con un N pequeño o cuando el problema es crítico y es muy fundamental encontrar la solución óptima sobre las demás y el tiempo de calculo no es un problema. Por el contrario, el algoritmo heurístico Greedy ofrece soluciones de gran calidad en tiempos sumamente reducidos (con una complejidad $O(N \log N)$), pero sin poder garantizar la obtención del óptimo global.

La visión sesgada del método greedy: Aunque en los dos primeros ejercicios el algoritmo greedy alcanzó la solución óptima de manera casi instantánea, el Ejercicio 3 demostró concretamente un caso donde este método falla en encontrar el óptimo. Al tomar decisiones irrevocables basadas pura y exclusivamente en el beneficio local inmediato (el mejor ratio), el algoritmo demostró carecer de una visión integral, llenando la mochila ineficientemente y dejando espacio ocioso que le impidió alcanzar la mejor combinación global.

8. Anexos: Código Fuente en Python

```
import time
import csv

# =====
# EJERCICIO 1: BÚSQUEDA EXHAUSTIVA (10 objetos, volumen)
# =====

def busqueda_exhaustiva(volumenes, valores, capacidad_max):
    n = len(volumenes)
    total_subconjuntos = 2 ** n

    # Estructura donde se guardan TODOS los subconjuntos generados
    todos_los_subconjuntos = []

    for numero in range(total_subconjuntos):
        objetos_incluidos = []
        volumen_total = 0
        valor_total = 0

        for i in range(n):
            bit = (numero >> i) & 1
            if bit == 1:
                objetos_incluidos.append(i)
                volumen_total += volumenes[i]
                valor_total += valores[i]

        todos_los_subconjuntos.append({
            "objetos": objetos_incluidos,
            "volumen": volumen_total,
            "valor": valor_total,
        })

    # Ahora que estan TODOS los subconjuntos generados y evaluados,
    # se pasa a una segunda estructura solo el mejor subconjunto valido.
    mejor_subconjunto = None
    for subconjunto in todos_los_subconjuntos:
```

```

        if subconjunto["volumen"] <= capacidad_max:
            if mejor_subconjunto is None or subconjunto["valor"] >
mejor_subconjunto["valor"]:
                mejor_subconjunto = subconjunto

    return mejor_subconjunto, todos_los_subconjuntos

# =====
# EJERCICIO 2: ALGORITMO GREEDY (10 objetos, volumen)
# =====

def algoritmo_greedy(volumenes, valores, capacidad_max):
    n = len(volumenes)

    # Se calcula el ratio $/cm3 de cada objeto
    objetos = []
    for i in range(n):
        ratio = valores[i] / volumenes[i]
        objetos.append({"indice": i, "volumen": volumenes[i], "valor": valores[i],
"ratio": ratio})

    objetos.sort(key=lambda obj: obj["ratio"], reverse=True)

    volumen_acumulado = 0
    valor_acumulado = 0
    seleccionados = []

    for obj in objetos:
        if volumen_acumulado + obj["volumen"] <= capacidad_max:
            seleccionados.append(obj["indice"])
            volumen_acumulado += obj["volumen"]
            valor_acumulado += obj["valor"]
        # si no entra, se descarta y se sigue probando con el siguiente

    resultado = {
        "objetos": seleccionados,
        "volumen": volumen_acumulado,
        "valor": valor_acumulado,

```

```

    }

    return resultado, objetos # objetos ya trae el ratio calculado y ordenado

# =====
# FUNCIONES AUXILIARES DE IMPRESION
# =====

def mostrar_subconjunto(nombre, subconjunto, offset_indices=1):
    objetos_mostrados = [i + offset_indices for i in subconjunto["objetos"]]
    print(f"{nombre}:")
    print(f"  Objetos incluidos : {objetos_mostrados}")
    print(f"  Volumen total      : {subconjunto['volumen']}")
    print(f"  Valor total        : {subconjunto['valor']}")

# =====
# FUNCIONES AUXILIARES DE EXPORTACION A CSV
# =====

def guardar_csv(nombre_archivo, encabezados, filas):
    with open(nombre_archivo, "w", newline="", encoding="utf-8") as archivo:
        escritor = csv.writer(archivo)
        escritor.writerow(encabezados)
        escritor.writerows(filas)
    print(f"  -> Guardado: {nombre_archivo}")

def exportar_subconjuntos_csv(nombre_archivo, todos_los_subconjuntos, capacidad_max,
mejor_subconjunto, offset_indices=1):
    encabezados = ["subconjunto_binario", "objetos", "volumen_total", "valor_total",
"valido", "es_optimo"]
    filas = []
    n = len(bin(len(todos_los_subconjuntos) - 1)) - 2 # cantidad de bits necesarios
    for indice, subconjunto in enumerate(todos_los_subconjuntos):
        objetos_mostrados = [i + offset_indices for i in subconjunto["objetos"]]
        texto_objetos = ", ".join(str(o) for o in objetos_mostrados) if
objetos_mostrados else "Ninguno"

```

```

        binario = format(indice, f"0{n}b")
        valido = subconjunto["volumen"] <= capacidad_max
        es_optimo = subconjunto is mejor_subconjunto
        filas.append([binario, texto_objetos, subconjunto["volumen"],
subconjunto["valor"], valido, es_optimo])
        guardar_csv(nombre_archivo, encabezados, filas)

def exportar_orden_greedy_csv(nombre_archivo, objetos_ordenados, seleccionados,
offset_indices=1):
    encabezados = ["orden", "objeto", "volumen", "valor", "ratio", "seleccionado"]
    filas = []
    seleccionados_set = set(seleccionados)
    for posicion, obj in enumerate(objetos_ordenados, start=1):
        filas.append([
            posicion,
            obj["indice"] + offset_indices,
            obj["volumen"],
            obj["valor"],
            round(obj["ratio"], 4),
            obj["indice"] in seleccionados_set,
        ])
    guardar_csv(nombre_archivo, encabezados, filas)

def exportar_comparacion_csv(nombre_archivo, filas_comparacion):
    encabezados = ["ejercicio", "metodo", "objetos", "capacidad_usada", "valor_total",
"tiempo_segundos", "alcanzo_optimo"]
    guardar_csv(nombre_archivo, encabezados, filas_comparacion)

# =====
# EJERCICIOS 1 y 2: 10 OBJETOS, RESTRICCION DE VOLUMEN
# =====

def ejercicios_1_y_2(filas_comparacion):
    print("=" * 60)
    print("EJERCICIOS 1 y 2 - 10 objetos - Capacidad 4200 cm3")
    print("=" * 60)

```

```

# Objeto 1..10 -> indices 0..9
volumenes = [150, 325, 600, 805, 430, 1200, 770, 60, 930, 353]
valores = [20, 40, 50, 36, 25, 64, 54, 18, 46, 28]
capacidad_max = 4200

# --- Búsqueda exhaustiva ---
inicio = time.perf_counter()
mejor_exhaustivo, todos = busqueda_exhaustiva(volumenes, valores, capacidad_max)
fin = time.perf_counter()
tiempo_exhaustivo = fin - inicio

print(f"\nCantidad de subconjuntos evaluados: {len(todos)} (2^10)")
mostrar_subconjunto("\nMejor solución - Búsqueda Exhaustiva", mejor_exhaustivo)
print(f" Tiempo de ejecución: {tiempo_exhaustivo:.6f} segundos")

# --- Algoritmo Greedy ---
inicio = time.perf_counter()
resultado_greedy, objetos_ordenados = algoritmo_greedy(volumenes, valores,
capacidad_max)
fin = time.perf_counter()
tiempo_greedy = fin - inicio

print("\nOrden de objetos según ratio valor/volumen (mayor a menor):")
for obj in objetos_ordenados:
    print(f" Objeto {obj['indice']+1}: volumen={obj['volumen']}, "
          f"valor={obj['valor']}, ratio={obj['ratio']:.4f}")

mostrar_subconjunto("\nSolución - Algoritmo Greedy", resultado_greedy)
print(f" Tiempo de ejecución: {tiempo_greedy:.6f} segundos")

# --- Comparación ---
print("\n--- Comparación Ejercicios 1 y 2 ---")
print(f"Valor óptimo (exhaustivo) : {mejor_exhaustivo['valor']}")
print(f"Valor obtenido (greedy) : {resultado_greedy['valor']}")
alcanzo_optimo = resultado_greedy["valor"] == mejor_exhaustivo["valor"]
if alcanzo_optimo:
    print("-> El greedy SI alcanza el valor óptimo en este caso.")
else:

```

```

    diferencia = mejor_exhaustivo["valor"] - resultado_greedy["valor"]
    print(f"-> El greedy NO alcanzo el optimo. Diferencia: {diferencia}")
    print(f"Tiempo exhaustivo: {tiempo_exhaustivo:.6f} s vs. Tiempo greedy:
{tiempo_greedy:.6f} s")

# --- Exportacion a CSV ---
print("\nExportando resultados a CSV...")
exportar_subconjuntos_csv("ejercicio1_subconjuntos.csv", todos, capacidad_max,
mejor_exhaustivo)
exportar_orden_greedy_csv("ejercicio2_orden_greedy.csv", objetos_ordenados,
resultado_greedy["objetos"])

filas_comparacion.append([
    "1 y 2", "Busqueda Exhaustiva",
    ", ".join(str(i + 1) for i in mejor_exhaustivo["objetos"]),
    mejor_exhaustivo["volumen"], mejor_exhaustivo["valor"],
round(tiempo_exhaustivo, 8), True,
])
filas_comparacion.append([
    "1 y 2", "Greedy",
    ", ".join(str(i + 1) for i in resultado_greedy["objetos"]),
    resultado_greedy["volumen"], resultado_greedy["valor"], round(tiempo_greedy,
8), alcanzo_optimo,
])

# =====
# EJERCICIO 3: 3 ELEMENTOS, RESTRICCION DE PESO
# =====

def ejercicio_3(filas_comparacion):
    print("\n" + "=" * 60)
    print("EJERCICIO 3 - 3 elementos - Capacidad 3000 grs")
    print("=" * 60)

    # Elemento 1..3 -> indices 0..2
    pesos = [1800, 600, 1200]
    valores = [72, 36, 60]
    capacidad_max = 3000

```

```

# --- Búsqueda exhaustiva ---
inicio = time.perf_counter()
mejor_exhaustivo, todos = busqueda_exhaustiva(pesos, valores, capacidad_max)
fin = time.perf_counter()
tiempo_exhaustivo = fin - inicio

print(f"\nCantidad de subconjuntos evaluados: {len(todos)} (2^3)")
for subconjunto in todos:
    objetos_mostrados = [i + 1 for i in subconjunto["objetos"]]
    texto_objetos = str(objetos_mostrados) if objetos_mostrados else "Ninguno"
    valido = "valido" if subconjunto["volumen"] <= capacidad_max else "excede
capacidad"
    print(f"  {texto_objetos:<12} "
          f"peso={subconjunto['volumen']:<6} valor={subconjunto['valor']:<5}
({valido})")

mostrar_subconjunto("\nMejor solucion - Búsqueda Exhaustiva", mejor_exhaustivo)
print(f"  Tiempo de ejecucion: {tiempo_exhaustivo:.8f} segundos")

# --- Algoritmo Greedy ---
inicio = time.perf_counter()
resultado_greedy, objetos_ordenados = algoritmo_greedy(pesos, valores,
capacidad_max)
fin = time.perf_counter()
tiempo_greedy = fin - inicio

print("\nOrden de elementos segun ratio valor/peso (mayor a menor):")
for obj in objetos_ordenados:
    print(f"  Elemento {obj['indice']+1}: peso={obj['volumen']}, "
          f"valor={obj['valor']}, ratio={obj['ratio']:.4f}")

mostrar_subconjunto("\nSolucion - Algoritmo Greedy", resultado_greedy)
print(f"  Tiempo de ejecucion: {tiempo_greedy:.8f} segundos")

# --- Comparacion ---
print("\n--- Comparacion Ejercicio 3 ---")
print(f"Valor optimo (exhaustivo) : {mejor_exhaustivo['valor']}")
print(f"Valor obtenido (greedy)   : {resultado_greedy['valor']}")

```

```

alcanzo_optimo = resultado_greedy["valor"] == mejor_exhaustivo["valor"]
if alcanzo_optimo:
    print("-> El greedy SI alcanzo el valor optimo en este caso.")
else:
    diferencia = mejor_exhaustivo["valor"] - resultado_greedy["valor"]
    print(f"-> El greedy NO alcanzo el optimo. Diferencia: {diferencia}")
    print("    Esto pasa porque el greedy elige por ratio valor/peso paso a paso")
    print("    y ese criterio local no siempre lleva a la mejor combinacion
global.")

# --- Exportacion a CSV ---
print("\nExportando resultados a CSV...")
exportar_subconjuntos_csv("ejercicio3_subconjuntos.csv", todos, capacidad_max,
mejor_exhaustivo)
exportar_orden_greedy_csv("ejercicio3_orden_greedy.csv", objetos_ordenados,
resultado_greedy["objetos"])

filas_comparacion.append([
    "3", "Busqueda Exhaustiva",
    ", ".join(str(i + 1) for i in mejor_exhaustivo["objetos"]),
    mejor_exhaustivo["volumen"], mejor_exhaustivo["valor"],
round(tiempo_exhaustivo, 8), True,
])
filas_comparacion.append([
    "3", "Greedy",
    ", ".join(str(i + 1) for i in resultado_greedy["objetos"]),
    resultado_greedy["volumen"], resultado_greedy["valor"], round(tiempo_greedy,
8), alcanzo_optimo,
])

# =====
# MAIN
# =====
if __name__ == "__main__":
    filas_comparacion = []

    ejercicios_1_y_2(filas_comparacion)
    ejercicio_3(filas_comparacion)

```

```
print("\n" + "=" * 60)
print("Exportando tabla comparativa general...")
exportar_comparacion_csv("comparacion_general.csv", filas_comparacion)
print("=" * 60)
```